

2-1. 프로토타입

프로퍼티는 객체 상태를 나타내는 값(Data)으로 Key:Value로 되어 있으며, 객체는 0개 이상의 프로퍼티로 이루어진 집합이다.

1. argument 객체

- JavaScript의 모든 함수 스코프에는 arguments라는 특별한 변수가 있다. 이 변수는 함수에 넘겨진 모든 인자에 대한 정보가 담겨 있다
- arguments 객체는 Array가 아니다. 물론 length 프로퍼티도 있고 여러모로 Array와 비슷하게 생겼지만 Array.prototype을 상속받지는 않았다
- arguments에는 push, pop, slice 같은 표준 메소드가 없다.
- 일반 for문을 이용해 순회는 할 수 있지만, Array의 메소드를 이용하려면 arguments를 Array로 변환해야 한다.
- arguments 변수는 Function 안에서 다시 정의할 수 없다.
- var 구문이나 파라미터에 arguments라는 이름으로 변수를 정의해도 변수가 재정의되지 않는다

```
function Person(first, last) {
    this.first = first;
    this.last = last;
}

// 함수 속성 정의
Person.prototype.fullname = function(joiner, options) {
    console.log("TRACE Person.prototype.fullname ");
    console.log("options param :: " + options);
    options = options || { order: "western" };
    console.log("options param update :: " + options);
    var first = options.order === "western" ? this.first : this.last;
    var last = options.order === "western" ? this.last : this.first;
    return first + (joiner || " ") + last;
};

// 함수 선언
Person.fullname = function() {
    // 결과: Person.prototype.fullname.call(this, joiner, ..., argN);
    console.log("TRACE Person.fullname :: " + arguments);
    return Function.call.apply(Person.prototype.fullname, arguments);
};

// 객체 생성
var grace = new Person("Grace", "Hopper");

console.log("call :: grace.fullname() =====");
// 함수 속성 호출
console.log(grace.fullname()); // 'Grace Hopper'

// 함수 선언 호출
console.log("call :: Person.fullname order western =====");
```

```

console.log(Person.fullname({ first: "Alan", last: "Turing" }, ", ", { order: "western" }));
console.log("call :: Person.fullname order null
=====");
console.log(Person.fullname({ first: "Alan", last: "Turing" }, ", "));
console.log("call :: Person.fullname order western
=====");
console.log(Person.fullname({ first: "Alan", last: "Turing" }, ", ", { order: "seoul" }));

```

```

call :: grace.fullname() =====
TRACE Person.prototype.fullname
options param :: undefined
options param update :: [object Object]
Grace Hopper
call :: Person.fullname order western =====
TRACE Person.fullname :: [object Arguments]
TRACE Person.prototype.fullname
options param :: [object Object]
options param update :: [object Object]
Alan, Turing
call :: Person.fullname order null =====
TRACE Person.fullname :: [object Arguments]
TRACE Person.prototype.fullname
options param :: undefined
options param update :: [object Object]
Alan, Turing
call :: Person.fullname order western =====
TRACE Person.fullname :: [object Arguments]
TRACE Person.prototype.fullname
options param :: [object Object]
options param update :: [object Object]
Turing, Alan

```

2. 프로토타입

- 객체의 원형인 프로토타입을 이용하여 새로운 객체를 만드는 프로그래밍 기법
- JavaScript의 모든 객체는 자신을 생성한 객체 원형에 대한 숨겨진 연결을 갖는다. 이때 자기 자신을 생성 하기 위해 사용된 객체원형을 프로토타입이라 함
- JavaScript의 모든 객체는 Object객체의 프로토타입을 기반으로 확장 되었기 때문에 이 연결의 끝은 Object객체의 프로토타입 Object임

```
> console.log(Object.getOwnPropertyDescriptors(Person));
```

```

▼ {length: {...}, name: {...}, arguments: {...}, caller: {...}, prototype: {...}, ...} i
  ▶ arguments: {value: null, writable: false, enumerable: false, configurable: false}
  ▶ caller: {value: null, writable: false, enumerable: false, configurable: false}
  ▶ fullname: {writable: true, enumerable: true, configurable: true, value: f}
  ▶ length: {value: 2, writable: false, enumerable: false, configurable: true}
  ▶ name: {value: 'Person', writable: false, enumerable: false, configurable: true}
  ▶ prototype: {value: {...}, writable: true, enumerable: false, configurable: false}
  ▼ [[Prototype]]: Object
    ▶ constructor: f Object()
    ▶ hasOwnProperty: f hasOwnProperty()
    ▶ isPrototypeOf: f isPrototypeOf()
    ▶ propertyIsEnumerable: f propertyIsEnumerable()
    ▶ toLocaleString: f toLocaleString()
    ▶ toString: f toString()
    ▶ valueOf: f valueOf()
    ▶ __defineGetter__: f __defineGetter__()
    ▶ __defineSetter__: f __defineSetter__()
    ▶ __lookupGetter__: f __lookupGetter__()
    ▶ __lookupSetter__: f __lookupSetter__()
    ▶ __proto__: (...)
    ▶ get __proto__: f __proto__()
    ▶ set __proto__: f __proto__()

```

이중 대괄호로 되어있는 [[Prototype]]이 보입니다.

모든 객체는 [[Prototype]]이라는 내부 슬롯을 가진다. 내부 슬롯은 자바스크립트 엔진의 내부 로직이므로 원칙적으로 접근할 수 없지만, [[Prototype]]은 __proto__를 통해 간접적으로 접근할 수 있다.

자바스크립트 엔진은 객체의 프로퍼티를 생성할 때, 기본값으로 프로퍼티 어트리뷰트를 자동 정의한다.

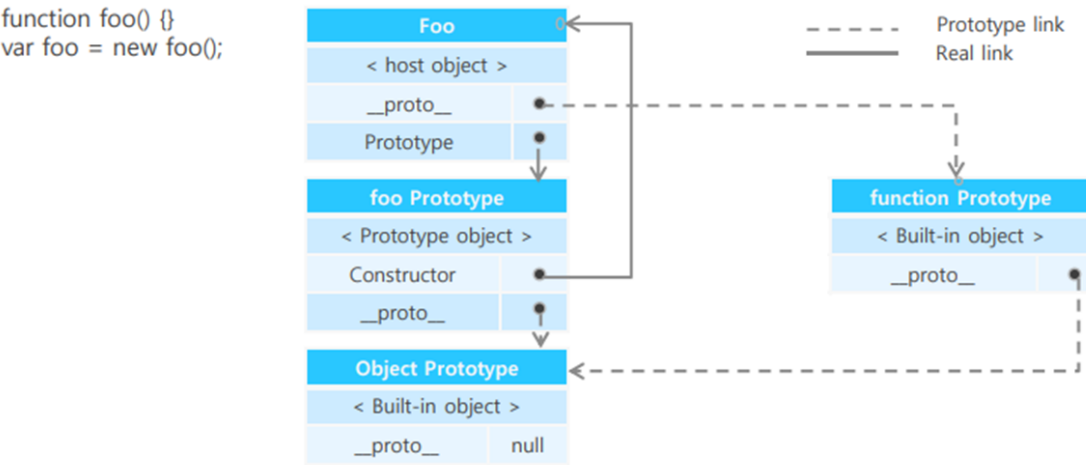
```
{% hint style="info" %}
```

- 내부 슬롯 : 자바스크립트 엔진의 구현 알고리즘을 설명하기 위해 ECMAScript 사양에서 사용하는 의사 프로퍼티 (pseudo property)
- 내부 메서드 : 위 설명과 동일한 의사 메서드 (pseudo method)
- ECMAScript 사양에 등장하는 이중 대괄호로 감싼 이름들이 내부 슬롯과 내부 메서드다.

자바스크립트 엔진에서 실제로 동작하지만, 개발자가 직접 접근할 수 있도록 공개된 프로퍼티는 아니다. {% endhint %}

Prototype property 란 ?

- 모든 함수 객체의 Constructor는 prototype이란 프로퍼티를 가지고 있다.
- 이 prototype property는 Object가 생성될 당시 만들어지는 객체 자신의 원형이 될 prototype 객체를 가리킨다



2-1. 프로퍼티 종류

데이터 프로퍼티 (Data Property)	접근자 프로퍼티 (Accessor Property)
키와 값으로 구성된 일반적인 프로퍼티	자체적으로 값을 갖지 않고 다른 데이터 프로퍼티의 값을 읽거나 저장할 때 호출되는 접근자 함수(Accssor Function)으로 구성된 프로퍼티

2-1-1. 데이터 프로퍼티 (Data Property)

자바스크립트 엔진이 프로퍼티를 생성할 때 기본값으로 자동 정의됩니다.

프로퍼티 어트리뷰트	프로퍼티 디스크립터 객체의 프로퍼티	설명
[[Value]]	value	프로퍼티 키를 통해 반환되는 프로퍼티 값 값을 변경할 때, [[Value]]를 재할당 하는 것. 프로퍼티가 없을 때는 동적 생성하고 저장한다.
[[Writable]]	writable	프로퍼티 값의 변경 가능 여부를 boolean 값으로 갖는다. false면, [[Value]]의 값을 변경할 수 없는 읽기 전용으로 만든다.
[[Enumerable]]	enumerable	프로퍼티의 열거 가능 여부를 boolean 값으로 갖는다. for...in 이나 Object.keys 사용 가능 여부가 된다.

		프로퍼티의 재정의 가능 여부를 boolean 값으로 갖는다.
[[Configurable]]	configurable	false인 경우, 해당 프로퍼티의 삭제, 프로퍼티 어트리뷰트 값의 변경이 금지된다.
		[[Writable]]이 True라면, [[Value]]의 변경과 [[Writable]]의 변경만 허용된다.

2-1-2. 접근자 프로퍼티 (Accessor Property)

접근자 프로퍼티에서는 우리가 흔히 아는 getter, setter 가 자동적으로 정의 됩니다. 다만 . getter, setter 모두 정의할 수 있고, 하나만 정의할 수도 있습니다

프로퍼티 어트리뷰트	프로퍼티 디스크립터	설명
[[Get]]	get	접근자 프로퍼티를 통해 데이터 프로퍼티 값을 읽을 때 호출되는 접근자 함수 접근자 프로퍼티 키로 프로퍼티 값에 접근하면 프로퍼티 어트리뷰트 [[Get]]의 값, 즉 getter 함수가 호출되고 그 결과가 프로퍼티 값으로 반환된다.
[[Set]]	set	접근자 프로퍼티를 통해 데이터 프로퍼티 값을 저장할 때 호출되는 접근자 함수 접근자 프로퍼티 키로 프로퍼티 값을 저장하면 프로퍼티 어트리뷰트 [[Set]]의 값, 즉 setter 함수가 호출되고 그 결과가 프로퍼티 값으로 저장된다.
[[Enumerable]]	enumerable	프로퍼티의 열거 가능 여부를 boolean 값으로 갖는다. for...in 이나 Object.keys 사용 가능 여부가 된다.
[[Configurable]]	configurable	프로퍼티의 재정의 가능 여부를 boolean 값으로 갖는다. false인 경우, 해당 프로퍼티의 삭제, 프로퍼티 어트리뷰트 값의 변경이 금지된다. [[Writable]]이 True라면, [[Value]]의 변경과 [[Writable]]의 변경만 허용된다.

```
const person = {
  // 데이터 프로퍼티
  firstName: '홍',
  lastName: '길동',
  // 접근자 프로퍼티 (접근자 함수로 구성)
  get fullName() {
    return `${this.firstName} ${this.lastName}`;
  },
  // 접근자 프로퍼티 (접근자 함수로 구성)
```

```

    set fullName(name) {
      [this.firstName, this.lastName] = name.split(' ');
    }
  };

  console.log(person.firstName + ' ' + person.lastName);
  // 홍길동

  person.fullName = '길동 김';
  console.log(person);
  // {firstName: '길동', lastName: '김'}

  console.log(person.fullName);
  // 길동 김

```

3. 예제

3-1. 예제 1

```

function foo(x) {
  this.x = x;
};
var A = new foo('hello');
console.log(A.x); // hello
console.log(foo.prototype.x); // undefined
console.log(A.prototype.x) // Cannot read property 'x' of undefined

```

A는 함수 객체가 아닌 foo라는 원형 함수 객체를 통해 만들어진 Object 객체에 확장된 단일 객체

A 객체가 생성 시점에 foo의 prototype.x가 없음

-> A는 prototype property를 소유하고 있지 않음

3-2. 예제 2

```

var A = function () {
  this.x = function () {
    console.log('hello');
  };
};
A.x = function() {
  console.log('world');
};
var B = new A();
var C = new A();
B.x(); // hello
C.x(); // hello

```

```
A.x(); // world
```

B,C 는 객체 생성시 A의 prototype Object를 사용함

prototype Object는 A가 생성될 시점 정보만 가지고 있음

A의 prototype Object가 알고 있는 x는 function () { console.log('hello');};임

A.x를 아무리 수정을 하여도 A의 prototype Object는 변경되지 않음

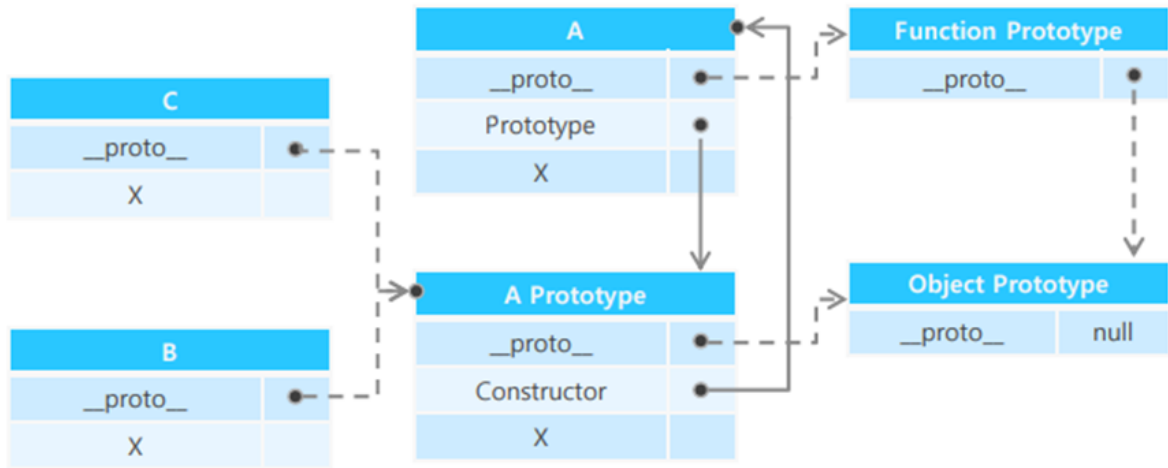
3-3. 예제 2

```
var A = function () { };
A.x = function() {
  console.log('hello');
};
A.prototype.x = function () {
  console.log('world');
};
var B = new A();
var C = new A();
B.x(); // world
C.x(); // world
A.x(); // hello
A.prototype.x = function () {
  console.log('world2');
};
B.x(); // world2
C.x(); // world2
```

B,C 는 객체 생성시 A의 prototype Object를 사용함

prototype Object는 A가 생성될 시점 정보만 가지고 있음

A의 prototype.x를 world로 선언 하였으므로 B,C는 A의 prototype을 따라감\



객체의 생성 과정에서 모태가 되는 프로토타입과의 연결고리가 이어져 상속관계를 통하여 상위 프로토타입으로 연속해서 이어지는 관계를 프로토타입 체인이라고 한다. 이 연결은 **proto** 를 따라 올라가게 된다

4. 공유와 상속

```

// B는 x를 상속
var A = function() {};
var B = new A();
A.prototype.x = 'hello';
console.log(A)           // [Function: A]
console.log(B)           // A {}
console.log(A.prototype.x); // hello
console.log(B.x);        // hello
B.x = 'world';
console.log(A) ;          // [Function: A]
console.log(B) ;          // A { x: 'world' }
console.log(A.prototype.x); // hello
console.log(B.x); // world
// B는 X를 공유
var C = function() {
  this.x = 'hello';
};
var D = new C();
console.log(D);           // { x: 'hello' }
console.log(C);           // [Function: C]
console.log(D.x);        // hello
console.log(C.x);        // undefined
  
```